INFORME TÉCNICO · ARQUITECTURA

Arquitectura de referencia de DocuProcess Cloud

Diagrama de componentes en AWS y justificación técnica RDS vs. DynamoDB

PROYECTO	DocuProcess Cloud
CURSO	Infraestructura III · Universidad ICESI
AUTORES	Daniela Castaño Moreno y Alejandro Pérez García
VERSIÓN	1.0 · 13 de noviembre de 2026
REPOSITORIO	github.com/Alejandroperezgarc1a/docuprocess-cloud
REGIÓN	us-east-1

NAVEGACIÓN

Contenido

Vista general 3

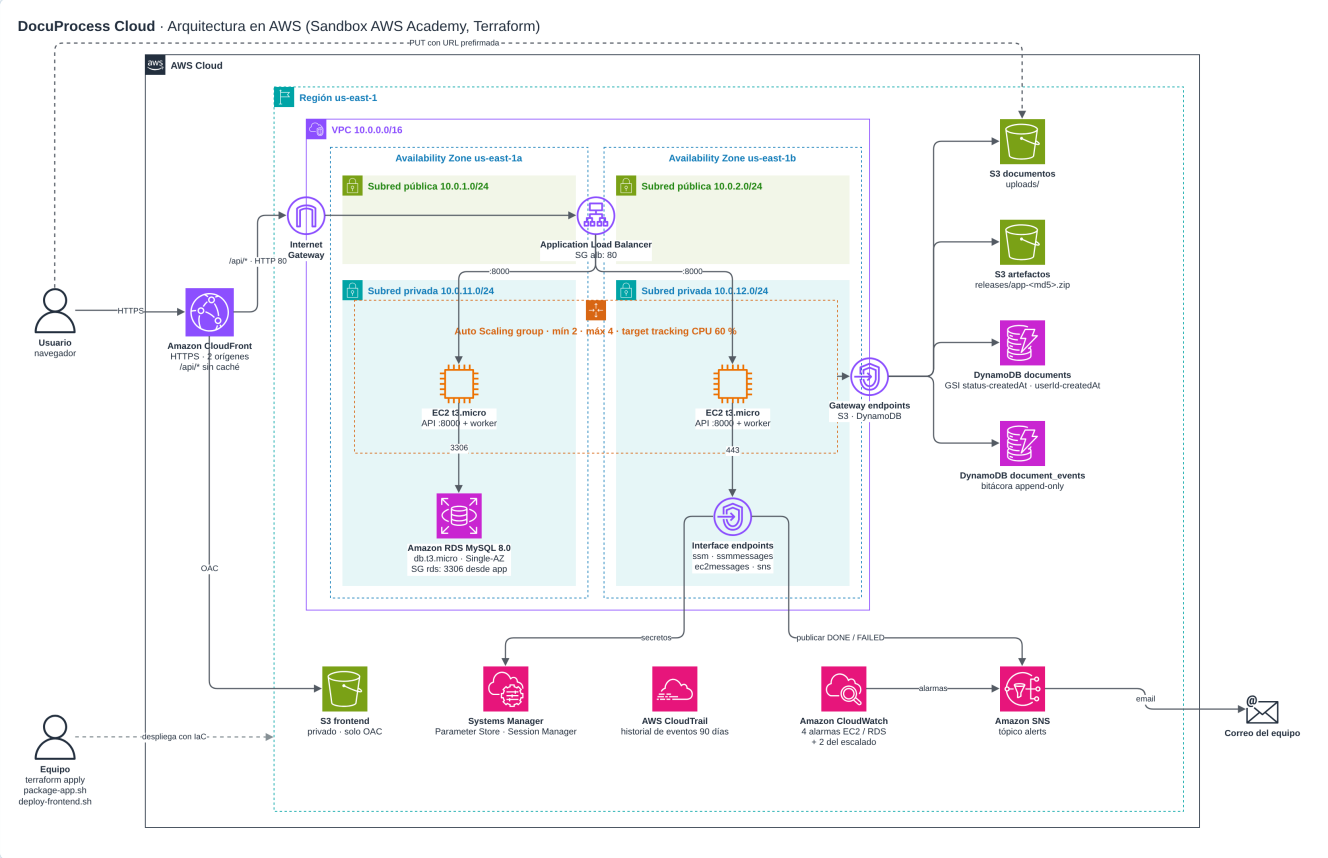
- Diagrama de arquitectura en AWS 3
- Diagrama de componentes 3
- Justificación del diseño técnico: RDS vs. DynamoDB 4

Los números de página corresponden al PDF. El documento fuente completo es docs/architecture.md.

> Este informe presenta el diagrama de arquitectura de DocuProcess Cloud en AWS y la justificación técnica del diseño de datos: RDS MySQL para cuentas y autenticación, DynamoDB para el estado, la cola y el historial de documentos, y S3 para los archivos. El documento fuente completo es docs/architecture.md.

Vista general

Diagrama de arquitectura en AWS



Vista con los íconos oficiales de AWS: región, VPC, las dos zonas de disponibilidad con sus subredes públicas y privadas, el Auto Scaling group y los servicios administrados. La fuente editable está en `img/docuprocess-aws.drawio` (se abre con draw.io / diagrams.net). El diagrama Mermaid de la sección siguiente muestra el mismo despliegue con el detalle de puertos y security groups.

Diagrama de componentes

Todo lo que aparece en el diagrama lo crea Terraform (`infra/`), salvo el usuario, el correo del equipo y el historial de eventos de CloudTrail, que AWS mantiene en cada cuenta sin recurso propio. Las líneas punteadas son tráfico de métricas o de subida directa.

Lectura del diagrama:

- **Un solo dominio público.** CloudFront sirve el frontend desde S3 (comportamiento por defecto, cacheado) y reenvía `/api/*` al ALB sin caché, con `Authorization` , `Content-Type` y `Origin` . El navegador siempre ve HTTPS; el tramo CloudFront → ALB es HTTP porque el Sandbox no permite dominio propio ni certificado ACM.
- **Subida directa.** El archivo no pasa por las instancias: el navegador hace `PUT` al bucket de documentos con la URL prefirmada que devuelve la API.

- **Red cerrada.** Las instancias y RDS viven en subredes privadas sin NAT ni IP pública. Todo el tráfico hacia AWS sale por endpoints: gateway para S3 y DynamoDB (gratis, rutas en la tabla privada) e interface para SSM y SNS (ENI en cada subred privada con private DNS). Los repositorios `dnf` de Amazon Linux 2023 también se alcanzan por el gateway de S3.
- **Security groups encadenados.** `alb` (80 desde internet) → `app` (8000 solo desde `alb`) → `rds` (3306 solo desde `app`). `endpoints` acepta 443 desde la VPC. No hay puerto 22: la administración es por Session Manager.
- **Arranque de cada instancia.** El userData lee los secretos de Parameter Store, descarga el artefacto `releases/app-<md5>.zip`, instala `nodejs24`, arranca los servicios systemd de la API y el worker, y solo se da por lista cuando `GET /api/ready` llega a MySQL.

Justificación del diseño técnico: RDS vs. DynamoDB

La plataforma usa **dos motores de base de datos** porque resuelven problemas distintos. La regla de diseño es: **RDS para lo que exige integridad relacional; DynamoDB para lo que exige acceso por clave y escala horizontal; S3 para los bytes**.

Necesidad del sistema	Motor elegido	Por qué
Cuentas, correos únicos y autenticación	**RDS MySQL 8.0** (`db.t3.micro`, Single-AZ)	Datos relacionales con unicidad, transacciones ACID y consultas por correo; el esquema cambia poco y el volumen es pequeño
Estado de cada documento y cola del worker	**DynamoDB** (`documents`, on-demand)	Acceso por clave (`documentId`) y por índice (`userId`, `status`); escala sin administrar servidores y la carga es intermitente
Historial auditable de cambios de estado	**DynamoDB** (`document_events`, append-only)	Eventos inmutables ordenados por `documentId` + `eventAt`; crecen sin migraciones ni joins
Archivos originales	**S3**	Objetos binarios de hasta 10 MB; no pertenecen a una base de datos

RDS MySQL: cuentas y autenticación

La tabla `users` necesita **unicidad del correo** (`UNIQUE(email)`), **transacciones ACID**, **consultas por atributo** (`WHERE email = ?`) e **integridad referencial** si se agregan tablas relacionadas. El correo es un atributo consultable, no una clave natural; en DynamoDB habría que mantener un ítem «correo → usuario» y coordinar dos escrituras por registro. Además, el login compara el hash de bcrypt después de buscar por correo: en SQL es una consulta trivial; en DynamoDB serían dos accesos por clave.

El esquema lo aplica la API al arrancar (`app/api/schema.sql`), no Terraform: Terraform crea la instancia, no las tablas. En el Sandbox la instancia es Single-AZ, `gp2` y `db.t3.micro`; el subnet group cubre dos AZ para poder pasar a Multi-AZ sin rehacer la red.

DynamoDB: documentos, cola y bitácora

Los documentos tienen tres patrones de acceso conocidos de antemano:

1. **Por documento:** `GetItem(documentId)` para el detalle y para que el worker lo reclame.
2. **Por usuario:** `Query` del GSI `userId-createdAt`, más recientes primero (lista del frontend).
3. **Por estado:** `Query` del GSI `status-createdAt` para que el worker tome los `PENDING` más antiguos (cola lógica).

La bitácora añade un cuarto patrón: **por documento en orden cronológico** (`Query(documentId)` ascendente). Los GSI `userId-eventAt` y `day-eventAt` permiten auditar a un usuario entre documentos o un día completo sin `Scan`.

DynamoDB encaja porque el estado del documento es **variable en la práctica**: `result` cambia según el tipo (páginas para PDF, ancho/alto para imágenes) y `error` solo existe en `FAILED`. En SQL serían columnas nulas o una tabla de atributos; en DynamoDB es un mapa opcional. La cola se resuelve con el GSI y un `UpdateItem` condicional (`#s = :pending`), sin bloquear filas ni abrir conexiones mientras se reclama. El historial crece de forma monotónica y el `PutItem` es O(1); una tabla de eventos en MySQL obligaría a administrar particiones, purgas y capacidad.

La tabla `document_events` es **append-only**: la política de recurso del rol no concede `UpdateItem` ni `DeleteItem` , así que la inmutabilidad la impone AWS y no solo el código. Estado y evento se escriben juntos con `TransactWriteItems` ; con `DDB_TRANSACTIONS=false` (respaldo del Sandbox) se escriben en secuencia.

Comparación lado a lado

Criterio	RDS MySQL	DynamoDB
Modelo	Relacional, SQL, joins	Clave-valor/documento, API por ítem
Esquema	Fijo, migraciones	Flexible; solo se declaran claves e índices
Unicidad	`UNIQUE` , `PRIMARY KEY`	Condiciones (`attribute_not_exists`) o claves diseñadas
Transacciones	ACID completas	`TransactWriteItems` / `TransactGetItems` (hasta 100 ítems)
Consistencia	Fuerte por defecto	Eventual en GSIs; `ConsistentRead` en la tabla base
Escala	Vertical (clase de instancia)	Horizontal (particiones)
Capacidad	Aprovisionada	On-demand o aprovisionada
Consultas ad hoc	Sí (`WHERE` , `JOIN` , `GROUP BY`)	No: hay que diseñar índices o usar `Scan` (caro)
Backups	Automáticos + snapshots	PITR (35 días) + on-demand
Seguridad de acceso	SG + usuario/contraseña + TLS	Políticas de recurso/IAM + cifrado en reposo
Operación	Parches y ventanas de RDS	Totalmente gestionado, sin servidores
Límite por ítem	Filas sin límite práctico	400 KB
Costo en la demo	Instancia `db.t3.micro` encendida 24/7	Por petición (céntimos)
Ajuste al problema	Cuentas y autenticación	Estado, cola e historial

Por qué no una sola base

Todo en RDS es viable a esta escala, pero acopla la carga del worker a la de autenticación: el worker sondea la cola cada 5 segundos desde 2 a 4 instancias y compite por CPU y conexiones con el login, que ya es la operación cara (bcrypt costo 12). La cola requeriría `SELECT … FOR UPDATE SKIP LOCKED` con conexiones abiertas, el `result` variable serían columnas nulas, la inmutabilidad de la bitácora dependería de `GRANT` s que la propia API administra y una caída de RDS (Single-AZ) detendría también el procesamiento.

Todo en DynamoDB obligaría a resolver la unicidad del correo con un ítem sintético y transacciones, a buscar el usuario por un atributo que no es clave y a renunciar a joins y consultas ad hoc. El volumen de usuarios de la demo no necesita escala horizontal y el enunciado pide RDS para cuentas y autenticación.

Consecuencia aceptada: una caída de RDS solo afecta registro y login; los documentos, la cola y el historial siguen disponibles. El health check del ALB (`/api/health`) no depende de MySQL, así que el ASG no recicla instancias por una caída de la base. Si el volumen creciera, el siguiente paso sería S3 → SQS → worker, manteniendo DynamoDB como fuente de verdad y el `UpdateItem` condicional como garantía de idempotencia.